WP-CLI

$ wp --hands-on

Manage, Automate and Extend WordPress

from the Terminal

Sudar Muthu
@Sudarmuthu

Chandra Patel
@chandrappatel

WP CLI

WordCamp Asia 2026

Welcome!

90 Minutes, Hands-On
Beginner-friendly with guided exercises

Questions Welcome
Interrupt us anytime - no need to wait

Live Demos Only
Every command runs in real time on screen

By the end of this session, you'll manage WordPress entirely from the terminal.

Pre-Workshop Setup

Quick verification:

```
$ wp --info
$ wp --version
```

Confirm these three essentials before we dive in:

- ✓ **Local WordPress instance**
  LocalWP, MAMP, Docker, or any local dev environment
- ✓ **Terminal / command line access**
  macOS Terminal, Windows PowerShell, or Linux shell
- ✓ **WP-CLI installed**
  Install guide: wp-cli.org/#installing

What is WP-CLI?

What is WP-CLI?

The Official WordPress CLI

Built by the WordPress community, endorsed by WordPress.org

No Browser Required

Manage plugins, themes, users and content from your terminal

Free & Open Source

Actively maintained with a vibrant contributor community

Runs on Any WordPress Host

Pre-installed on most managed WordPress hosting platforms

wp `<command>` — that's all it takes.

A superpower for anyone who manages WordPress sites.

WP-CLI in Action

Update Plugins & Themes
`wp plugin update --all`

Search & Replace URLs
`wp search-replace 'old.com' 'new.com'`

Export & Import Database
`wp db export backup.sql`

Manage Users & Roles
`wp user list --role=administrator`

Run Cron Jobs On-Demand
`wp cron event run --due-now`

Automate Repetitive Tasks
Use in shell scripts

One command. Seconds saved. Multiply that across 100 sites.

That's the real power of WP-CLI.

How WP-CLI Works?

The Anatomy of a WP-CLI Command

Every command follows this pattern:

`wp`	`post`	`list`	`ID`
binary	*command*	*subcommand*	*argument*

`--prompt`
flag

`wp` — The entry point

Every WP-CLI command starts here — no exceptions

`command` — The object

What you're managing: `plugin` `post` `user` `db`

`subcommand` — The action

What to do: `install` `update` `delete` `list`

`[arguments]` — What to act on

The target name, ID, or slug. Optional on bulk ops

`[--flags]` — Optional modifiers

Control output, skip prompts, format results:
`--prompt` `--url` `--skip-plugins`

WP-CLI Commands in Action

bash — wp-cli workshop

```
$ wp plugin list                                    // Lists all plugins with status
$ wp plugin install woocommerce --activate          // Installs and activates plugin
$ wp theme activate twentytwentyfour                // Switch the active theme
$ wp user update 1 --user_pass=newpassword          // Resets the admin password
```

■ wp ■ command ■ argument ■ --flag ■ value

The `--path` Flag

How WP-CLI locates WordPress

1. **Current directory**
   Run wp from inside your WP folder

2. **Parent directories**
   Walks up the tree until wp-config.php

3. **`--path` flag**
   Explicit override — any directory

Usage patterns

method 1 — navigate to WP root first

```
# Navigate to your WordPress folder first
$ cd ~/mysite
$ wp plugin list
```

OR

method 2 — specify --path from anywhere

```
# No cd needed — specify the path directly
$ wp plugin list --path=~/mysite
# Works from any directory!
```

💡 **Pro tip:** Use `--path` in scripts and cron jobs where the working directory is not guaranteed.

Getting Help — Built Right In

Every WP-CLI command ships with its own documentation — no browser needed.

Drill down from general → specific

```
$ wp help
```
GLOBAL

Lists every available WP-CLI command

```
$ wp help plugin
```
GROUP

Shows all subcommands for a group

```
$ wp help plugin install
```
CMD

Full docs: synopsis, flags, examples

```
$ wp plugin install --help
```
ALIAS

Same docs — append anywhere

What you see in the terminal

wp help plugin install

```
NAME
  wp plugin install

SYNOPSIS
  wp plugin install <plugin>
    [--activate]
    [--force]

OPTIONS
  --activate
  # ... (output continues)
```

Most Used Commands

Real-World Workflows

Fresh Site in less than 5 minute

From empty folder to working WordPress — four commands.

1

Download Core

`wp core download`

Downloads WordPress files into the current directory

2

Create Config

`wp config create`

Writes wp-config.php with your database credentials

3

Create Database

`wp db create`

Creates the MySQL database defined in wp-config.php

4

Install WordPress

`wp core install`

Completes the famous 5-minute install in seconds

SCRIPT

```
$ wp core download && wp config create && wp db create && wp core install ...
```

Local Development Sync

Pull production content into your local environment — safely and repeatably.

PRODUCTION

mysite.com

• LIVE

1. `wp db export prod.sql`
   Dump production database to file
2. `wp db import prod.sql`
   Load dump into local database
3. `wp search-replace mysite.com local.dev`
   Rewrite all prod URLs to local (use --all-tables)
4. `wp plugin deactivate --all`
   Turn off prod-only plugins (caching, CDN)
5. `wp option delete siteurl home`
   Clear stale option cache entries

LOCAL

local.dev

• DEV

⚠ **Never run wp db import on production.** Always verify --path points to your local install first.

Dos & Don'ts

Best Practices

1 Always Backup

```
$ wp db export prod.sql
```

Before any destructive operation

2 Use --dry-run

```
$ wp s-r ... --dry-run
```

Preview changes before applying

3 Read the Docs

```
$ wp help post create
```

Before any unfamiliar command

4 Use --path in Scripts

```
$ wp cron run --path=.
```

Avoid ambiguous working directory

5 Keep WP-CLI Updated

```
$ wp cli update
```

Get new commands and security fixes

Treat WP-CLI like production infrastructure — reliable, auditable, repeatable.

Common Mistakes

1. Don't run as `root` unless necessary — use `--allow-root` only when you understand the implications
2. Don't skip `--dry-run` on `search-replace` — always preview before committing changes
3. Don't run `wp plugin update --all` on production without a backup
4. Don't run `wp db import` without verifying the SQL file first
5. Don't run untested commands on production — test locally or on staging first

AI + WP-CLI

Three Ways AI Makes WP-CLI Better

1
Learn Commands

2
Run Commands

3
Build Commands

Learning WP-CLI with AI

Ask AI for WP-CLI Commands

You don't need to **memorise** every command — just **describe what you want**:

> *How do I reset the admin password without sending a notification email?*
>
> *Find all posts with a specific meta key, list them, then delete them*

The pattern: What you want + context + constraints or caveats

Pro Tip: Always verify AI-generated commands before running on production — copy, review, then execute

Prompt Patterns Cheat Sheet

Goal	Add to your prompt
Safety first	...and show me how to preview it first
Want explanation	...and explain each flag
For a script	...that works in a bash loop for multiple sites
Troubleshooting	...to diagnose or fix [problem]
Multisite	..on a multisite, targeting subsite at [url]

Running WP-CLI With AI

From Generating Commands to Running Them

You ask → AI decides what to run → AI runs it → AI tells you the result

YOU

How many published posts do I have?

→

AI RUNS

```
wp --path=~/Sites/wplab post list --post_status=publish --format=count
```

→

RESULT

You have 47 published posts.

No copy-paste required - AI runs the command and returns the answer in natural language

Building Custom WP-CLI Commands With AI

Why Build Custom WP-CLI Commands?

Built-in WP-CLI covers most tasks but sometimes you need site-specific automation:

- Run your own database cleanup routine with one command
- Audit content that follows your specific business rules
- Automate a workflow unique to your theme or plugin

With AI: Describe your custom command in plain English and AI scaffolds the PHP class, registers it, and writes the logic

Anatomy of a Custom WP-CLI Command

1. **`Register with WP_CLI::add_command()`**

   Maps command name to a PHP class or callable

2. **`PHPDoc comment = auto-generated help text`**

   ## DESCRIPTION, ## OPTIONS, ## EXAMPLES become wp help output

3. **`__invoke( $args, $assoc_args ) runs the logic`**

   Use WP_CLI::success(), WP_CLI::error(), WP_CLI::line() for output

Full guide: make.wordpress.org/cli/handbook/guides/commands-cookbook/

Wrap Up

What We Covered

WP-CLI Fundamentals

- Command anatomy, installation, getting help
- Core, plugin, theme, user, post, db, config commands
- Real-world workflows: setup, local development sync

AI + WP-CLI

- Learn commands — ask AI in plain English
- Run commands — AI executes WP-CLI against your local site
- Build commands — AI scaffolds custom PHP from a description

Where to Go From Here

WP-CLI Official Resources

- Official docs: developer.wordpress.org/cli/commands/
- Handbook: make.wordpress.org/cli/handbook/
- Package index: wp-cli.org/package-index/

Useful Community Packages

- wp-cli/profile-command — profile slow hooks and queries
- wp-cli/doctor-command — check site health from the terminal

Thank You